# Designing a Universal Information Language

Oscar Bender-Stone

March 26, 2026

DRAFT

| Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 32 | 20 | Space | 48 | 30 | 0 | 64 | 40 | @ | 80 | 50 | P | 96 | 60 | ` | 112 | 70 | p |
| 33 | 21 | ! | 49 | 31 | 1 | 65 | 41 | A | 81 | 51 | Q | 97 | 61 | a | 113 | 71 | q |
| 34 | 22 | " | 50 | 32 | 2 | 66 | 42 | B | 82 | 52 | R | 98 | 62 | b | 114 | 72 | r |
| 35 | 23 | # | 51 | 33 | 3 | 67 | 43 | C | 83 | 53 | S | 99 | 63 | c | 115 | 73 | s |
| 36 | 24 | $ | 52 | 34 | 4 | 68 | 44 | D | 84 | 54 | T | 100 | 64 | d | 116 | 74 | t |
| 37 | 25 | % | 53 | 35 | 5 | 69 | 45 | E | 85 | 55 | U | 101 | 65 | e | 117 | 75 | u |
| 38 | 26 | & | 54 | 36 | 6 | 70 | 46 | F | 86 | 56 | V | 102 | 66 | f | 118 | 76 | v |
| 39 | 27 | ' | 55 | 37 | 7 | 71 | 47 | G | 87 | 57 | W | 103 | 67 | g | 119 | 77 | w |
| 40 | 28 | ( | 56 | 38 | 8 | 72 | 48 | H | 88 | 58 | X | 104 | 68 | h | 120 | 78 | x |
| 41 | 29 | ) | 57 | 39 | 9 | 73 | 49 | I | 89 | 59 | Y | 105 | 69 | i | 121 | 79 | y |
| 42 | 2A | * | 58 | 3A | : | 74 | 4A | J | 90 | 5A | Z | 106 | 6A | j | 122 | 7A | z |
| 43 | 2B | + | 59 | 3B | ; | 75 | 4B | K | 91 | 5B | [ | 107 | 6B | k | 123 | 7B | { |
| 44 | 2C | , | 60 | 3C | < | 76 | 4C | L | 92 | 5C | \ | 108 | 6C | l | 124 | 7C | \| |
| 45 | 2D | - | 61 | 3D | = | 77 | 4D | M | 93 | 5D | ] | 109 | 6D | m | 125 | 7D | } |
| 46 | 2E | . | 62 | 3E | > | 78 | 4E | N | 94 | 5E | ^ | 110 | 6E | n | 126 | 7E | ~ |
| 47 | 2F | / | 63 | 3F | ? | 79 | 4F | O | 95 | 5F | _ | 111 | 6F | o | | | |

DRAFT

# Agenda

**Introduction**

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

# Background

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]
    - Can be a *process*, or *knowledge*
    - Most useful as *thing*
  - **Floridi:** Information is **truthful**[3]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]
    - Can be a *process*, or *knowledge*
    - Most useful as *thing*
  - **Floridi:** Information is **truthful**[3]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

# Problems

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). “The sum of all human knowledge”: A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

# Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

# Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

# Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data
  - Example: Wikipedia[2]

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

# Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data
  - Example: Wikipedia[2]
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**
  - **Interoperable:** a portable data format
  - Example: US health records[3]

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

# Existing Solutions

DRAFT

[1]Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2]Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3]Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

DRAFT

# Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — Not standardized
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine

DRAFT

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

— Hard-coded, proprietary rules

DRAFT

# Proposed Solution

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

# Proposed Solution

- **Welkin**: an information language

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
- **Goals:**

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
- **Goals:**
  - **Universality:** be completely expressive

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof

DRAFT

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof
  - **Scalability:** work at any size

DRAFT

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof
  - **Scalability:** work at any size
    - Be efficient

DRAFT

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof
  - **Scalability:** work at any size
    - Be efficient
    - Support: future optimizations
    - Describe improvements *in* Welkin

DRAFT

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

DRAFT

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof
  - **Scalability:** work at any size
    - Be efficient
    - Support: future optimizations
    - Describe improvements *in* Welkin
  - **Standardization:** be completely *unambiguous*

DRAFT

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

# Proposed Solution

- **Welkin**: an information language
  - Pronounced: "VEL-KIN"
  - Old German word, meaning "cloud", "sky"[1]
- **Goals:**
  - **Universality:** be completely expressive
    - *Any* computer program
    - *Any* "reliable" proof
  - **Scalability:** work at any size
    - Be efficient
    - Support: future optimizations
    - Describe improvements *in* Welkin
  - **Standardization:** be completely *unambiguous*
    - Have a formal specification

[1] Oxford English Dictionary (2025), s.v. "welkin, n."

DRAFT

# Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

# General Requirements

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program
   - Need effective operations

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid empty symbols!

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid empty symbols!
   - Symbol Grounding Problem[2]

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid empty symbols!
   - Symbol Grounding Problem[2]

- Achieve both through **representations**

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

# Representations

DRAFT

## Representations

- Centered around *representing* things
- $a - c \to b$: $a$ **represents** $b$ **in context** $c$

DRAFT

## Representations

- Centered around *representing* things
- $a - c \to b$: $a$ **represents** $b$ **in context** $c$
  - $a$: **sign**

DRAFT

## Representations

- Centered around *representing* things
- $a - c \to b$: $a$ **represents** $b$ **in context** $c$
  - $a$: **sign**
  - $b$: **referent**

DRAFT

## Representations

- Centered around *representing* things
- $a - c \to b$: $a$ **represents** $b$ **in context** $c$
  - $a$: **sign**
  - $b$: **referent**
  - $c$: **context**

DRAFT

## Representations

- Centered around *representing* things
- $a - c \to b$: $a$ **represents** $b$ **in context** $c$
  - $a$: **sign**
  - $b$: **referent**
  - $c$: **context**
- Track *faithfulness* of representations through **truth**

DRAFT

# Units

DRAFT

# Units

- **Finitely defined**

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Representations are units

DRAFT

# Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Representations are units
- Each unit has its own **context**

DRAFT

# Information

- Representations

DRAFT

## Information

- Representations $\sim$ **Truth**
- Information

DRAFT

## Information

- Representations ~ **Truth**
- Information ~ **Proofs**
-

DRAFT

## Rules

- **Internal Transitivity:**
- **Contextual Lifting:**

DRAFT

## Rules

- **Internal Transitivity:**
- **Contextual Lifting:**
- **Lattice Laws:**
  - Units as sets

DRAFT

# Turing Completeness

DRAFT

## Turing Completeness

- Part of Universality
- **Theorem:**

DRAFT

# Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - $\varphi$: some mapping between units and programs
  - Interpretation:

DRAFT

# Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - $\varphi$: some mapping between units and programs
  - Interpretation: $a - c \to b$ iff $\varphi_c(\langle\varphi_a\rangle) = \langle\varphi_b\rangle$
  - $\langle\varphi_a\rangle$:

DRAFT

# Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - $\varphi$: some mapping between units and programs
  - Interpretation: $a - c \to b$ iff $\varphi_c(\langle\varphi_a\rangle) = \langle\varphi_b\rangle$
  - $\langle\varphi_a\rangle$: encodes $\varphi_a$ as a string
- Later:

DRAFT

## Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - $\varphi$: some mapping between units and programs
  - Interpretation: $a - c \to b$ iff $\varphi_c(\langle\varphi_a\rangle) = \langle\varphi_b\rangle$
  - $\langle\varphi_a\rangle$: encodes $\varphi_a$ as a string
- Later: completeness for **proofs**!

DRAFT

# Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

# Overview

DRAFT

# Encoding

- Characters written in `type-writer font`[1]
- Define US-ASCII: Printable (codes: 32-126) + `EOF` (code: 127)
- **Token:** collection of contiguous characters
  - Simplifies grammar

DRAFT

[1]Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

# US-ASCII

| Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph | Dec. | Hex. | Glyph |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 32 | 20 | Space | 46 | 2E | . | 60 | 3C | < | 74 | 4A | J | 88 | 58 | X | 102 | 66 | f | 116 | 74 | t |
| 33 | 21 | ! | 47 | 2F | / | 61 | 3D | = | 75 | 4B | K | 89 | 59 | Y | 103 | 67 | g | 117 | 75 | u |
| 34 | 22 | " | 48 | 30 | 0 | 62 | 3E | > | 76 | 4C | L | 90 | 5A | Z | 104 | 68 | h | 118 | 76 | v |
| 35 | 23 | # | 49 | 31 | 1 | 63 | 3F | ? | 77 | 4D | M | 91 | 5B | [ | 105 | 69 | i | 119 | 77 | w |
| 36 | 24 | $ | 50 | 32 | 2 | 64 | 40 | @ | 78 | 4E | N | 92 | 5C | \ | 106 | 6A | j | 120 | 78 | x |
| 37 | 25 | % | 51 | 33 | 3 | 65 | 41 | A | 79 | 4F | O | 93 | 5D | ] | 107 | 6B | k | 121 | 79 | y |
| 38 | 26 | & | 52 | 34 | 4 | 66 | 42 | B | 80 | 50 | P | 94 | 5E | ^ | 108 | 6C | l | 122 | 7A | z |
| 39 | 27 | ' | 53 | 35 | 5 | 67 | 43 | C | 81 | 51 | Q | 95 | 5F | _ | 109 | 6D | m | 123 | 7B | { |
| 40 | 28 | ( | 54 | 36 | 6 | 68 | 44 | D | 82 | 52 | R | 96 | 60 | ` | 110 | 6E | n | 124 | 7C | \| |
| 41 | 29 | ) | 55 | 37 | 7 | 69 | 45 | E | 83 | 53 | S | 97 | 61 | a | 111 | 6F | o | 125 | 7D | } |
| 42 | 2A | * | 56 | 38 | 8 | 70 | 46 | F | 84 | 54 | T | 98 | 62 | b | 112 | 70 | p | 126 | 7E | ~ |
| 43 | 2B | + | 57 | 39 | 9 | 71 | 47 | G | 85 | 55 | U | 99 | 63 | c | 113 | 71 | q | | | |
| 44 | 2C | , | 58 | 3A | : | 72 | 48 | H | 86 | 56 | V | 100 | 64 | d | 114 | 72 | r | | | |
| 45 | 2D | - | 59 | 3B | ; | 73 | 49 | I | 87 | 57 | W | 101 | 65 | e | 115 | 73 | s | | | |

Table 1: Printable US-ASCII codes and glyphs.

DRAFT

# Character Classes

| Set Name | Characters |
|---|---|
| PRINTABLE | Listed in Table 1 |
| WHITESPACE | \t \| \r \| Space |
| DELIMITER | { \| } \| ' \| " \| . \| , |
| RESERVED | DELIMITER \| * \| @ |
| SQ_CHAR | Any PRINTABLE except ' and \ |
| DQ_CHAR | Any PRINTABLE except " and \ |
| ID_CHAR | Any PRINTABLE not in RESERVED or WHITESPACE |

Table 2: Key character classes.

DRAFT

# Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

# Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

# Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:**

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

# Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:** consume tokens, not characters

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

# Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:** consume tokens, not characters
  - Simplifies grammar

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

# The Welkin Grammar

```
start := units - lexeme -> EOF,
units := "" | {unit - lexeme -> *{"",
"," - lexeme -> units}},
unit := node - lexeme -> *{
  ":=" - lexeme -> choices,
  *{"", arc - lexeme -> chain}
},
choices := *{"", "|"} - lexeme ->
choices_list,
choices_list := *{"",
  chain - lexeme -> *{"", "|" - lexeme
-> choice_list}
},
chain := *{"", "*"} - seq -> path -
lexeme -> *{"", arc - lexeme ->
chain},
arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -
> "->",
other_arc := "<-" - seq -> node - seq
-> *{"-", "->"},
node := *{"", "*"} - seq -> path,
path := *{"", modifiers} - seq ->
trailer - seq -> *{"", graph},
modifiers := *{
  "@",
  "~" - seq -> *{"", "@"}
},
trailer := segment - seq -> *{"", "."
- seq -> trailer},
segment := HANDLE - lexeme -> *{"",
graph},
graph :=  "{" - lexeme -> units -
lexeme -> "}",
HANDLE := ID | STRING,
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR,
"\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR,
'\"'} - seq_many_till -> '"',
ID := ID_CHAR - seq_many_till ->
*{RESERVED, WHITESPACE}
```

Listing 1: Full Welkin grammar.

DRAFT

# Outline of Unambiguity

- Simple grammar class: LL(1)

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing[1]

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing[1]
- For our parsers, need Edelmann et. al:

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing[1]
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing[1]
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets
  - Need: certain sets are djsoint

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing[1]
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets
  - Need: certain sets are djsoint
  - For details, refer to thesis

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

# Agenda

Introduction

Language Overview

Syntax

**Metatheory**

Conclusion

DRAFT

# Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

# Comparisons to Existing Solutions

DRAFT

# Future Work

DRAFT